MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY
DEPARTMENT OF ITS



FPT UNIVERSITY

GRADUATE THESIS REPORT

Chatbot for answering historical questions using KG-RAG

MAJOR: ARTIFICIAL INTELLIGENCE

SUPERVISOR :**MSc. Do Thai Giang**

STUDENT 1:**Le Do Huy**
STUDENT 2:**Nguyen Hai Dang**
STUDENT 3:**Nguyen Duc Huy**

HA NOI, DECEMBER 2024

Acknowledgement

We wish to express our deepest gratitude to our supervisor, MSc. Do Thai Giang, for his guidance, invaluable insights, and unwavering support throughout the course of this research. His guidance has been instrumental in shaping this thesis.

Special appreciation goes to our dedicated classmate from AI1701, whose collaboration and commitment added significant value to this research. We are also deeply grateful to them for their encouragement, understanding, and shared moments of inspiration that have sustained us throughout this journey.

This thesis is a testament to the collective efforts of everyone who has supported us in this academic endeavor, and we are sincerely thankful for each contribution, no matter how big or small.

Abstract

Understanding and answering historical questions is crucial for preserving and disseminating cultural knowledge. This thesis introduces Graph-RAG (Graph Retrieval-Augmented Generation), an innovative system designed to answer historical questions, with a particular focus on Vietnamese historical data. The system constructs graphs in which nodes represent key concepts, events, or entities, while edges are defined based on embedding-based similarity and the strength of relationships between nodes, effectively capturing the complex interconnections within historical data. Using a self-developed dataset—the DHH-Vietnamese History Dataset, derived from 15 volumes of Vietnamese History by the Hanoi Institute of History, the grade 12 Vietnamese history textbook, and additional internet sources—Graph-RAG achieves significant advancements. Quantitative evaluation demonstrates that the system enhances answer accuracy and provides more reliable numerical data compared to baseline methods. These results highlight the system's potential to facilitate historical research and education in the Vietnamese context, offering precise, context-aware answers to complex queries about Vietnam's rich and diverse history.

Keywords: Graph-RAG; Vietnamese history; embeddings; key concepts; historical question answering

Contents

I Introduction . . . 4
1.1 Background . . . 4
1.2 Objectives and contribution . . . 4
1.3 Organization . . . 5
II Related work . . . 6
2.1 Question Answering Models . . . 6
2.1.1 Extractive Question Answering Models . . . 6
2.1.2 Generative Question Answering Models . . . 7
2.2 RAG-Based Methods . . . 9
2.3 Knowledge Graph RAG-Based Methods . . . 10
III Methodology . . . 13
3.1 Data and preprocessing . . . 13
3.1.1 Data . . . 13
3.1.2 Preprocessing . . . 14
3.2 System Design and Implementation . . . 16
3.2.1 Vietnamese Embeddings . . . 17
3.2.2 Document Processor . . . 19
3.2.3 KnowledgeGraph . . . 19
3.2.4 Query Engine . . . 20
IV Experimental result . . . 23
4.1 Metrics . . . 23
4.2 Training and Inference Setup . . . 23
4.2.1 Hardware . . . 23
4.2.2 Training duration . . . 23
4.2.3 Tunning hyperparameters . . . 24
V Results and Discussion . . . 25
5.1 Token Efficiency Analysis Across Hyperparameter Configurations . . . 25
5.2 Hyperparameter Sensitivity Analysis . . . 26
5.3 Token Efficiency vs. Quality Tradeoff . . . 27
5.4 Quantitative Comparison of Models . . . 28
VI Conclusion and Improvement . . . 30
6.1 Conclusion . . . 30
6.2 Improvement . . . 30

I Introduction

1.1 Background

Answering historical questions, especially those related to Vietnamese history, requires a nuanced understanding of diverse and interconnected events, entities, and relationships. Traditional natural language processing (NLP) methods often struggle with the complexity of historical queries, which demand both contextual reasoning and the integration of fragmented information. Simple text-based retrieval systems [1] [2] frequently fail to capture the multidimensional relationships between historical facts, leaving gaps in understanding and explanation.

Recent advancements in Retrieval-Augmented Generation (RAG) systems have introduced new possibilities by combining retrieval-based methods with generative models. These systems retrieve relevant knowledge to inform language models, improving both accuracy and contextual relevance in responses. However, historical data presents unique challenges due to its inherent complexity, including overlapping timelines, causality, and geographical or thematic connections.

Graph-RAG builds upon the RAG framework by incorporating graph-based representations of knowledge. In this approach, a graph is constructed where nodes represent key concepts, events, or entities, and edges are formed using similarity measures derived from embeddings and key concepts. This graph structure captures both local and global relationships within the data, allowing for more effective retrieval of contextually relevant information.

By applying Graph-RAG to Vietnamese historical data, this system provides a tailored solution to historical question answering. Vietnamese history, characterized by rich cultural, political, and social events spanning centuries, serves as an ideal domain to showcase the capabilities of Graph-RAG. With its ability to align information through embeddings and key concept similarity, Graph-RAG bridges the gap between fragmented historical records and the need for coherent, contextually accurate answers. This innovative approach not only enhances historical understanding but also paves the way for improved reasoning in domain-specific applications.

1.2 Objectives and contribution

The objective of this study is to address and mitigate the weaknesses in Knowledge Graph-based Retrieval-Augmented Generation (KG-RAG) systems, as shown in section 2.3 below, by enhancing the interpretation of user queries to avoid misinterpretation, improving the alignment of retrieved knowledge with user demands, expanding and regularly updating Knowledge Graphs to cover less common and emerging topics, optimizing the integration process with Large Language Models (LLMs) to maintain coherence in responses, boosting computational efficiency and scalability for deployment in resource-constrained environments, and balancing the trade-off between cost and accuracy to achieve higher accuracy. By tackling these challenges, the study aims to unlock the full potential of KG-RAG systems for applications ranging from customer service to advanced research assistance.

In this thesis, our approach brings some key contributions:

- We implement an integration of graph-based knowledge representation with retrieval-augmented generation to address complex historical question answering. The framework leverages graph structures to enhance the contextual understanding of entity relationships and improves the relevance of retrieved information.

- Enhanced Graph Representation Learning: We develop a tailored graph encoding mechanism that refines the embedding of historical entities and events, ensuring accurate and semantically enriched retrieval during query processing.

- Evaluation on Historical Question-Answering Tasks: We conduct extensive experiments on a custom benchmark dataset derived from Vietnamese history, showcasing the effectiveness of Graph-RAG.

1.3 Organization

The remainder of this thesis is structured as follows. In Section 2, we provide a concise overview of several related theories. Section 3 provides a deep insight into dataset, data preprocessing and augmentation, along with implementations using Graph-RAG. Section 4 demonstrates our results. We conclude the thesis while outlining potential avenues for future research.

II Related work

2.1 Question Answering Models

2.1.1 Extractive Question Answering Models

Extractive QA models, such as BERT[3], RoBERTa [4], and SpanBERT [5], have demonstrated remarkable advancements in question answering tasks by leveraging the Transformer architecture. These models operate on the principle of identifying the span of text in a given context that directly answers a query. By fine-tuning pre-trained language models on QA datasets like SQuAD, they achieve high precision in extracting explicit answers while maintaining efficient performance across various domains.

The workflow of extractive QA models begins with tokenizing the input, which includes both the question and the context. These tokens are embedded into dense vector representations that capture semantic relationships, which are then passed through a series of bidirectional self-attention layers in the Transformer architecture. Self-attention mechanisms allow the model to dynamically weigh the relevance of each token in the context with respect to the query. This process enables the model to construct a probabilistic map of the start and end positions of the answer span. Figure 1 illustrates this process, highlighting both the workflow and the token-level attention heatmap.

The objective function of extractive QA models is to maximize the conditional probability of the start and end tokens of the answer span, denoted as (a_s, a_e), given the context C and the query Q. Mathematically, this is represented as:

$$P(a_s, a_e|C, Q) = \textit{softmax}(h_{\textit{start}}(C, Q)) \cdot \textit{softmax}(h_{\textit{end}}(C, Q)), \tag{1}$$

where $h_{\text{start}}(C, Q)$ and $h_{\text{end}}(C, Q)$ are the logits produced by the Transformer for the start and end tokens, respectively. The softmax function normalizes these logits to represent probabilities over the entire token sequence, ensuring that the most probable span is selected.

One of the critical strengths of these models lies in their ability to generalize across domains by utilizing pre-trained representations from large-scale corpora. This pre-training equips the model with a strong understanding of language, including syntactic and semantic nuances, enabling it to perform well with moderate fine-tuning on domain-specific QA datasets. Additionally, these models excel in situations where the answer is explicitly present in the context, achieving state-of-the-art performance in benchmarks like SQuAD. Their capacity to handle varying question formulations further underscores their robustness in practical applications.

However, extractive QA models are not without limitations. Their reliance on extracting spans directly from the input context restricts their applicability to scenarios where the answer must be synthesized, inferred, or sourced from external knowledge. Furthermore, these models face challenges with long documents that exceed the maximum token limit (commonly 512 tokens) imposed by Transformer architectures. In such cases, truncating the context may result in the omission of critical information, significantly degrading performance. Strategies such as sliding window techniques or hierarchical processing have been proposed to mitigate this issue, but they introduce additional computational overhead.

Another limitation is their inability to incorporate external knowledge effectively. Unlike generative models that can retrieve and synthesize information from large knowledge bases or external sources, extractive QA models operate within the confines of the provided input. This shortcoming makes them less suitable for multi-hop reasoning or answering out-of-context

questions. Despite these challenges, the fine-grained attention mechanisms employed by these models provide valuable insights into how they process and prioritize information.

Figure 1 showcases an example of token-level attention, where the model focuses on the most relevant portions of the context to pinpoint the answer span. The workflow demonstrates the flow of inputs (context and question) through the Transformer layers to predict the start and end positions of the answer span. The heatmap highlights token-level attention scores, showing how the model identifies relevant tokens in the context for span prediction.

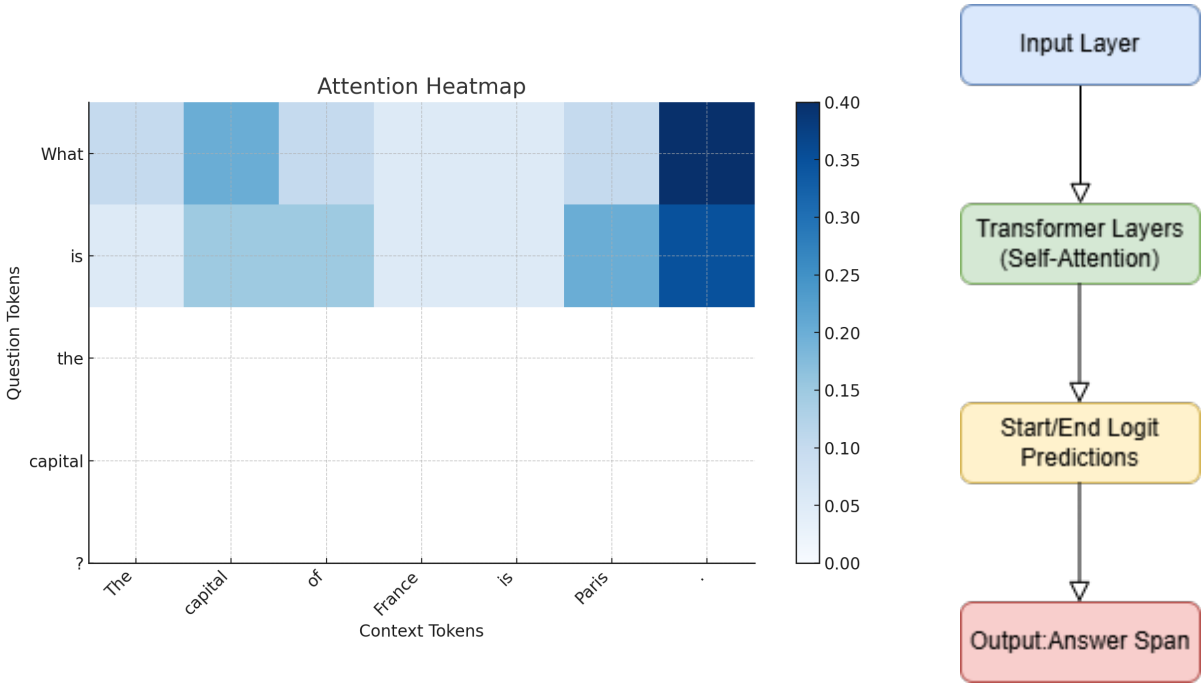


Figure 1: Extractive QA workflow and attention visualization.

In conclusion, extractive QA models represent a significant step forward in the automation of information retrieval and comprehension. Their ability to locate precise answers within text has profound implications for applications ranging from chatbots to information retrieval systems. However, addressing their limitations, particularly in handling long contexts and integrating external knowledge, remains an active area of research.

2.1.2 Generative Question Answering Models

Generative models such as GPT-3 [6], T5 [7], and BART [8] excel in generating free-text responses based on provided contexts. These models leverage architectures like encoder-decoder and autoregressive transformers, combining extensive pre-training on diverse datasets with fine-tuning for specific tasks, as shown in Figure 2. Pre-training involves predicting masked tokens or next-word sequences across large corpora, enabling models to learn syntactic, semantic, and contextual representations. Fine-tuning refines these capabilities for particular applications, such as open-domain question answering, summarization, and translation.

Their design facilitates remarkable flexibility, handling a range of question types from fact-based retrieval to open-ended reasoning. Notably, these models exhibit cross-domain generalization capabilities, synthesizing plausible responses even from ambiguous or incomplete inputs. For example, GPT-3's autoregressive approach generates text by predicting one token

at a time, while T5 and BART's encoder-decoder architecture translates input contexts into outputs, benefiting tasks like summarization or translation where understanding input relationships is critical.

However, these models face significant challenges. One major issue is hallucination, where outputs may be factually incorrect or unsupported by the input context. This problem arises from overgeneralization in training or insufficient grounding during inference. Additionally, processing long or multi-document contexts becomes inefficient due to token limit constraints, especially for complex queries requiring broad contextual understanding. Resource demands pose another limitation; these models require substantial computational power and memory, which increases costs and limits accessibility for smaller-scale applications.

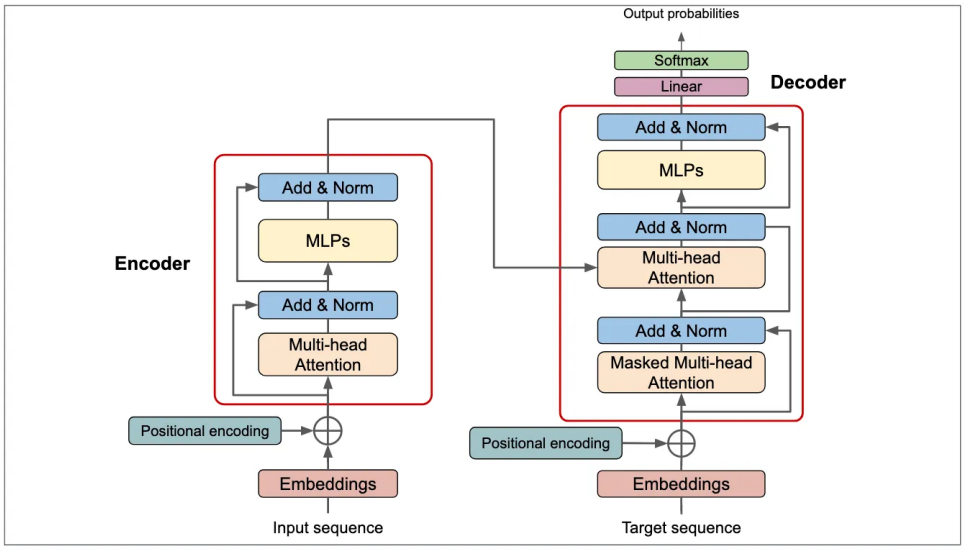


Figure 2: Illustration of a generative question-answering pipeline featuring an encoder-decoder structure [9]

To address these challenges, retrieval-augmented approaches such as REALM and RAG [10] integrate external knowledge retrieval mechanisms into generative pipelines. REALM (Retrieval-Augmented Language Model) is a model that integrates external knowledge retrieval into the generative process. During inference, REALM dynamically fetches relevant documents from a large corpus of text. This approach helps the model to have access to updated and specific information, thereby enriching the input and enhancing the factual accuracy of the generated responses. The integration of retrieval mechanisms allows REALM to answer questions more accurately by utilizing external knowledge sources. Meanwhile, RAG (Retrieval-Augmented Generation) combines a retriever module and a generative module to synthesize responses. The retriever module identifies pertinent passages from a large set of documents, while the generative module uses these passages to generate the final answer. This hybrid system ensures that the model can access and utilize the most relevant information available, improving the quality and accuracy of the responses. This capability is particularly impactful in domains requiring up-to-date or highly specific knowledge, such as medical information or technical support.

Reinforcement learning with human feedback (RLHF) [11] offers another solution by aligning model outputs with human preferences. WebGPT[12] exemplifies the use of RLHF by iteratively refining its responses based on human evaluations. In this approach, humans provide feedback on the generated responses, which is used to optimize the model through reward-based learning. By continuously incorporating human preferences, WebGPT aligns its

outputs with what humans find satisfactory and coherent. This method has proven effective in reducing hallucinations, where the model generates false or misleading information, and in improving the overall coherence of the responses. The iterative process ensures that the model learns from human feedback, making it more reliable and user-friendly.

Generative models also benefit from advancements in fine-tuning frameworks like PrimeQA's retrieval-augmented generation (RAG) pipeline [13]. The framework leverages modular components, including retrievers and generative readers, to enhance performance across various tasks. The retrievers identify relevant documents or passages from a large corpus, while the generative readers synthesize the final answers. This modularity allows for flexibility and adaptability in handling different types of queries. By combining retrieval and generation, PrimeQA demonstrates how hybrid architectures can balance accuracy, efficiency, and scalability. The framework is designed to be adaptable, enabling improvements and updates to individual components without disrupting the overall system.

Despite their limitations, generative models represent a significant paradigm shift in natural language understanding and generation. These models are capable of engaging in nuanced and flexible interactions, making them valuable for a wide range of applications. In conversational AI, generative models enable more natural and human-like interactions, improving user experience and satisfaction. In summarization, these models can distill complex information into concise and coherent summaries, aiding in information dissemination and comprehension. Moreover, with continued innovations such as retrieval augmentation, RLHF, and efficient fine-tuning pipelines, generative models are poised to overcome current challenges and expand their impact across diverse domains, including healthcare, education, and multilingual communication. These advancements promise to enhance the accessibility and effectiveness of generative models in real-world applications, driving progress in various industries.

2.2 RAG-Based Methods

RAG (Retrieval-Augmented Generation) baseline, introduced by Lewis et al. (2020)[14], combines a Dense Passage Retriever (DPR) with a generative model like BART. The process begins with the retrieval component, which starts with a corpus of documents (e.g., text documents, websites, databases). When a query is received, the DPR searches this corpus to find the most relevant documents, using vector embeddings to measure the semantic similarity between the query and the documents. The DPR retrieves the top-K documents that are most relevant to the query. These documents are then concatenated to form a context for the generative model.

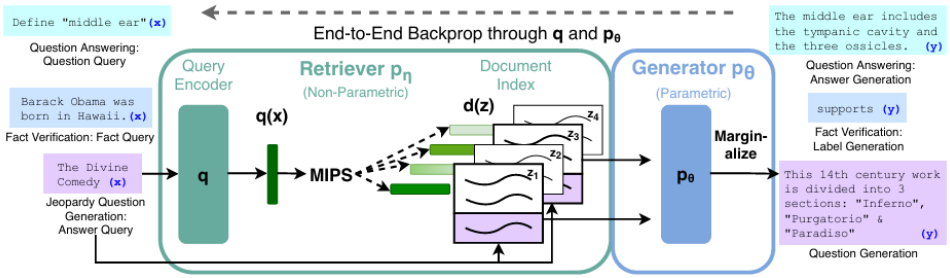


Figure 3: Overview RAG approach.

The concatenated documents are fed into a generative model like BART, which uses this context to generate a response to the query. The model leverages its pre-trained knowledge

and the retrieved context to produce a coherent and relevant answer. The strengths of RAG include efficient dense retrieval, ensuring that only the most relevant documents are retrieved. By using vector embeddings, the model ensures that the retrieved documents are semantically relevant to the query. The generative model's pre-trained knowledge allows it to generalize well across different domains. Additionally, the RAG model's modular design makes it easy to update individual components (e.g., the retriever or the generator) without needing to redesign the entire system. Figure 3 shows an overview of RAG approach, where it combines a pre-trained retriever (Query Encoder + Document Index) with a pre-trained seq2seq model (Generator) and fine-tune end-to-end. For query x, use Maximum Inner Product Search (MIPS) to find the top-K documents zi. For final prediction y, treat z as a latent variable and marginalize over seq2seq predictions given different documents.

However, the RAG model faces limitations. The context size is limited by the token capacity of the generative model, which can restrict the amount of information that can be considered at once. The generative model typically generates a response in a single pass, limiting the ability to iteratively refine the response based on additional context or feedback. Practical considerations for RAG include the need for an effective chunking strategy to ensure that relevant information is captured without overwhelming the model. Care must be taken to preserve important information during retrieval and generation to avoid losing critical details. Techniques such as re-ranking retrieved documents or using multiple passes for generation can help mitigate information loss.

Since the introduction of Retrieval-Augmented Generation (RAG) models, several notable versions have emerged, including REALM (Retrieval-Augmented Language Model) [10], FiD (Fusion-in-Decoder) [12], Atlas [15],... These RAG models have demonstrated significant strengths in their approach to question answering by combining retrieval with generation to ground answers in external content, thereby reducing the occurrence of hallucinations [10] [15].

One of the key advantages of RAG models is their adaptability through modular components, which allow for easy updates to the knowledge sources without requiring complete model retraining [10]. This makes them highly flexible and efficient for various applications. Additionally, RAG models offer scalability for large-scale open-domain question answering (QA) tasks, making them suitable for handling vast amounts of information and complex queries [10] [15].

Still, these models also face notable challenges. One significant issue is the increased latency due to real-time retrieval operations, which can slow down the response time [10]. Another challenge is the performance limitations based on the quality of the retrieval process; if the retrieved information is not relevant or accurate, the model's performance can be adversely affected [10]. Furthermore, advanced variants like FiD and Atlas require substantial computational resources, which may restrict accessibility for smaller research organizations or institutions with limited computational power [10] [15].

Despite all these, the continuous advancements in RAG models hold promise for improving the efficiency and effectiveness of question answering systems, making them a valuable tool in the field of natural language processing.

2.3 Knowledge Graph RAG-Based Methods

Knowledge Graph Retrieval-Augmented Generation (KG-RAG) models were introduced by Diego Sanmartin [16]. This innovative approach merges the structured knowledge embedded

in Knowledge Graphs (KG) with the generative capabilities of Large Language Models (LLM). KG-RAG aims to enhance the accuracy, relevance, and contextuality of AI-generated responses by leveraging the rich, interconnected data stored in Knowledge Graphs [16]. One of the key strengths of KG-RAG is its ability to reduce the occurrence of hallucinations—incorrect or nonsensical information—often generated by LLMs when they rely solely on their training data [16]. By integrating Knowledge Graphs, which provide a well-organized and reliable source of factual information, KG-RAG ensures that the responses are more grounded in reality [16]. The basic overview are shown in Figure 4

The mathematical foundation for this approach can be expressed as follows: Let $R(q)$ be the response generated for a query q by an LLM, and $K(q)$ be the knowledge extracted from the Knowledge Graph relevant to the query. The enhanced response $R_{KG}(q)$ can be modeled as:

$$R_{KG}(q) = \alpha \cdot R(q) + \beta \cdot K(q) \tag{2}$$

where α and β are weighting factors that balance the contribution of the LLM and the Knowledge Graph, respectively. This formula ensures that the final output incorporates both the generative capabilities of the LLM and the factual accuracy of the Knowledge Graph.

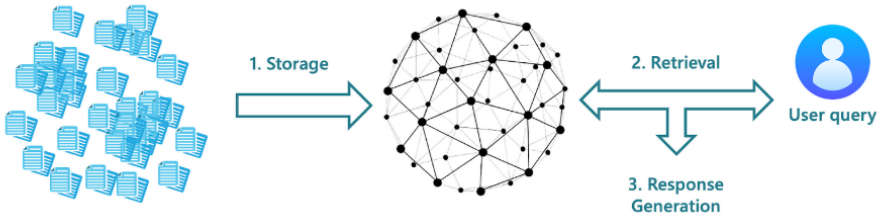


Figure 4: Diagram illustrating the KG-RAG framework [17]

Additionally, this hybrid approach excels at handling complex queries that require a deep understanding of context and relationships between entities [16]. The Knowledge Graph aids in disambiguating terms and providing relevant connections that an LLM might overlook [16]. For instance, if a query involves multiple entities with ambiguous references, the Knowledge Graph can clarify these relationships by providing structured data on the entities and their connections.

Another advantage noted is the framework's prompt-aware context generation [16]. KG-RAG can dynamically retrieve and incorporate relevant information from the Knowledge Graph in real-time, tailoring responses to the specific needs of the user's query [16]. This capability is particularly beneficial for applications that require precise and contextually rich information. For example, in educational tools, KG-RAG can provide detailed explanations and context-specific examples that enhance learning outcomes. In customer service bots, it can offer accurate and context-aware responses that improve user satisfaction. Similarly, in research assistants, KG-RAG can assist with complex data analysis and interpretation by providing well-grounded information and insights [16].

While KG-RAG offers several advantages, it is not without its weaknesses [16]. A study by Agrawal et al. (2024) reveals several critical challenges that KG-RAG systems face [18]. One of the primary issues is the accurate interpretation of the context of user queries [18]. Misinterpretation of context can lead to incorrect relation mapping and ineffective ambiguity resolution [18]. For example, if a query involves polysemous words or phrases, KG-RAG might struggle to accurately disambiguate these terms without sufficient context, leading to irrelevant or incorrect responses.

Furthermore, these systems sometimes fail to align the retrieved knowledge with the specific demands of the query [18]. This misalignment can result in less reliable answers, as the system might provide information that is tangentially related but not directly relevant to the user's needs [18]. For instance, if a user queries about a specific medical condition, the system might retrieve general information about the condition but miss crucial details relevant to the user's specific question.

Another significant challenge is the inherent limitations of Knowledge Graphs themselves [18]. Knowledge Graphs may not cover all possible domains comprehensively. They are often curated with a specific focus, which can result in gaps in knowledge for less common or emerging topics [18]. Additionally, the information within Knowledge Graphs can become outdated, leading to the dissemination of obsolete or incorrect data [18]. This issue is particularly problematic in rapidly evolving fields such as technology and medicine, where staying current is essential.

The integration process of Knowledge Graphs with LLMs also presents technical hurdles [18]. Ensuring seamless integration that maintains the flow of conversation while effectively leveraging the structured data from the Knowledge Graph is a complex task. Inaccurate integration can disrupt the coherence and naturalness of the generated responses, making interactions feel disjointed or artificial.

Moreover, there's the challenge of computational efficiency and scalability [18]. KG-RAG systems require significant computational resources to manage the real-time retrieval and integration of knowledge from Knowledge Graphs. This requirement can limit the deployment of KG-RAG in environments with constrained resources or where low-latency responses are critical.

These weaknesses highlight the need for ongoing improvements in KG-RAG systems to ensure they can consistently deliver accurate and contextually relevant responses [18]. Future advancements could focus on enhancing context interpretation algorithms, expanding and regularly updating Knowledge Graphs, improving integration techniques, and optimizing computational efficiency. Addressing these challenges will be crucial for realizing the full potential of KG-RAG systems in various applications, from customer service to advanced research assistants.

III Methodology

3.1 Data and preprocessing

3.1.1 Data

The data foundation for this Graph-RAG project is anchored in the grade 12 history textbook, a critical resource that outlines key historical developments of both Vietnam and the world from the early 20th century to 2000. To ensure a comprehensive and nuanced dataset, this primary source was supplemented with additional academic materials, including historical archives, scholarly analyses, and credible secondary sources.The grade 12 history textbook is divided into two major parts:

- Modern World History (1945–2000): This section contextualizes Vietnam's history within global developments and is organized into six chapters. Topics include the post-World War II world order, the Cold War and its impact, the decolonization of Asia, Africa, and Latin America, the economic and political resurgence of the United States, Western Europe, and Japan, international relations during and after the Cold War, and the scientific-technological revolution alongside globalization trends.

- Vietnamese History (1919–2000): This section offers a detailed account of Vietnam's historical trajectory, divided into five chapters. It begins with the national democratic movements from 1919 to 1930, continues through revolutionary activities leading to the August Revolution (1945) and the establishment of the Democratic Republic of Vietnam, and proceeds with Vietnam's resistance against French colonialism (1945–1954), the struggle against American intervention (1954–1975), and finally post-war reconstruction and reform (1975–2000).We also supplement the textbook with additional sources to add depth and contextual richness to the data.

Notably, we incorporate Episodes 8–12 of History of Vietnam, authored by Vietnam Academy of Social Sciences, Institute of History, and Vu Duy Men, which provide a detailed account of Vietnam's history from 1919 to 1965. These episodes offer in-depth perspectives on key events, such as the national democratic movements, revolutionary activities during the colonial period, the August Revolution, and the struggle against French colonialism, as well as the early phases of resistance against American intervention. This integration enriches the dataset by blending the structured content of the textbook with nuanced scholarly interpretations.

To further diversify the dataset, we draw upon supplementary materials from sources such as Wikipedia and other reputable academic and historical references. These sources contribute broader perspectives, additional details, and interpretations that complement the structured narrative of the textbook and scholarly accounts.

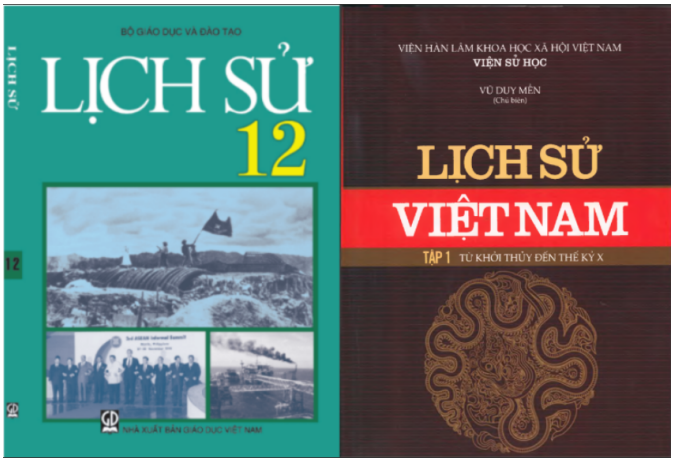


Figure 5: Vietnamese grade 12 history textbook and History of Vietnam

3.1.2 Preprocessing

The preprocessing stage is a critical component in preparing the historical data for integration into the Graph-RAG model. The goal of this stage is to ensure that the data, derived primarily from the grade 12 history textbook and supplemented by additional scholarly sources, is clean, consistent, and appropriately structured for model input. The preprocessing process adheres to several key procedures, aimed at enhancing data quality and ensuring that the final dataset is both accurate and usable for downstream tasks.

Giải quyết khó khăn về tài chính
Sự uổng rỗng của các kho bạc sau khi chính quyền cách mạng đã về tay nhân dân buộc Chính phủ lâm thời phải đề ra các giải pháp có tính cấp bách.

Giải pháp đầu tiên là quyên góp trong nhân dân. Ngày 4-9-1945, Chính phủ ra sắc lệnh số 4/SL quy định lập Quỹ Độc lập nhằm "thu nhận các món tiền và đồ vật của nhân dân sẵn lòng quyên giúp Chính phủ để ủng hộ nền độc lập của quốc gia".
Để khắc phục tình trạng trống rỗng về ngân sách, Chính phủ kêu gọi tinh thần tự nguyện đóng góp của nhân dân cả nước. Hưởng ứng cuộc vận động xây dựng "Quỹ độc lập", phong trào "Tuần lễ vàng" do Chính phủ phát động, nhân dân ta hăng hái đóng góp tiền của, vàng, bạc ủng hộ nền độc lập của Tổ quốc.
"Tuần lễ vàng" (từ ngày 16-9-1945) được tổ chức nhằm kêu gọi các nhà giàu đóng góp một phần tài sản cho đất nước vừa giành được độc lập còn gặp nhiều khó khăn.
Đông đảo nhân dân trong cả nước và một số kiều bào ở nước ngoài đã đăng ký hưởng ứng Quỹ Độc lập và Tuần lễ vàng. Có những nhà tư sản đóng góp hàng trăm, hàng chục lạng vàng. Có những người dân bình thường chỉ có chiếc nhẫn hoặc đôi khuyên tai cũng hăng hái đóng góp. Ngay cả ở Nam Bộ cũng có nhiều người tham gia đóng góp và tìm cách chuyển ra bằng được số vàng đó cho Chính phủ trong điều kiện chiến tranh đã diễn ra.
Quỹ Độc lập và Tuần lễ vàng đã thu được tất cả là 370kg vàng và 20 triệu đồng 1. Ngoài tiền và vàng, nhiều người còn hiến cả nhà cửa, ruộng vườn, xí nghiệp cho Chính phủ. Sự đóng góp tự nguyện xuất phát từ lòng yêu nước, từ lòng kính trọng Chủ tịch Hồ Chí Minh, từ sự lo lắng đến vận mệnh đất nước, đã giúp Chính phủ trang vải được một phần tài chính trong giai đoạn đầu. Ông Lê Văn Hiến nhận xét: "Nhìn cuộc sống và việc làm của cán bộ và chiến sĩ, dân rất thương, rất quý, nên dân đóng góp rất hăng hái... Nếu như hồi đó dân thấy cán bộ sống phè phỡn, ăn trên ngồi gốc hoặc sử dụng công quỹ bừa bãi, lãng phí thì làm sao mà dân hăng hái tự nguyện đóng góp được? Cho nên thanh liêm, không tham ô, không lãng phí là những điều kiện rất quan trọng không những đối với chi mà còn hết sức quan trọng đối với thu nữa".

Figure 6: Data enrichment

The first step in the preprocessing process involves the removal of extraneous elements from the raw data. This includes eliminating images, headlines, footnotes, references, and lesson-specific questions that are not directly relevant to the historical content. Such elements are typically found in the textbook and supplementary materials but do not contribute to the informational content required for the Graph-RAG model. The removal of these non-content elements ensures a clean, text-only dataset focused on the historical narrative. Additionally, any formatting inconsistencies such as incorrect spacing, errant line breaks, and misaligned text are rectified during this stage.

Once the extraneous content has been removed, the next step involves a comprehensive grammar check. This is essential to ensure that the text is grammatically correct and adheres to academic standards, enhancing readability and clarity. In addition to grammatical corrections, proper spacing and punctuation are applied to maintain a consistent structure throughout the dataset. This process ensures that the text is both professionally formatted and suitable for input into a machine learning model, which requires consistent linguistic structure to function effectively.

The core of the dataset is derived from the grade 12 history textbook, which provides a structured overview of Vietnamese history. Rather than independently segmenting the content, supplementary sources—such as Episodes 8–12 of History of Vietnam by Vietnam Academy of Social Science and other academic sources—are integrated within the framework of the textbook (Figure 6). This integration enriches the existing content, offering deeper insights into key historical events, figures, and movements while preserving the textbook's original structure. For instance, sections of the textbook discussing the August Revolution (1945) are supplemented with additional details from external scholarly sources, providing a more nuanced understanding of the event. This approach ensures that the enriched dataset remains aligned with the educational structure of the textbook while benefiting from the expanded historical context provided by the supplementary materials.

Following the cleaning and integration processes, the final dataset undergoes a thorough validation procedure to ensure both historical accuracy and internal consistency. The integrated data is cross-referenced with multiple authoritative sources, including academic publications, reputable online encyclopedias like Wikipedia, and primary historical documents. This step ensures that all information presented is factually accurate and appropriately contextualized within the broader scope of Vietnamese history. Furthermore, any discrepancies or inconsistencies identified during the validation process are addressed, resulting in a refined dataset that is both reliable and comprehensive.

The preprocessing stage ensures that the dataset is not only cleaned and formatted to meet academic standards but also enriched with diverse scholarly insights. By maintaining the structure of the grade 12 history textbook while integrating additional sources, the dataset is transformed into a comprehensive and coherent resource that is well-suited for use in the Graph-RAG model. This structured approach facilitates the generation of accurate and contextually rich responses to queries related to Vietnamese history, ensuring the model's effectiveness in historical data retrieval and analysis.

3.2 System Design and Implementation

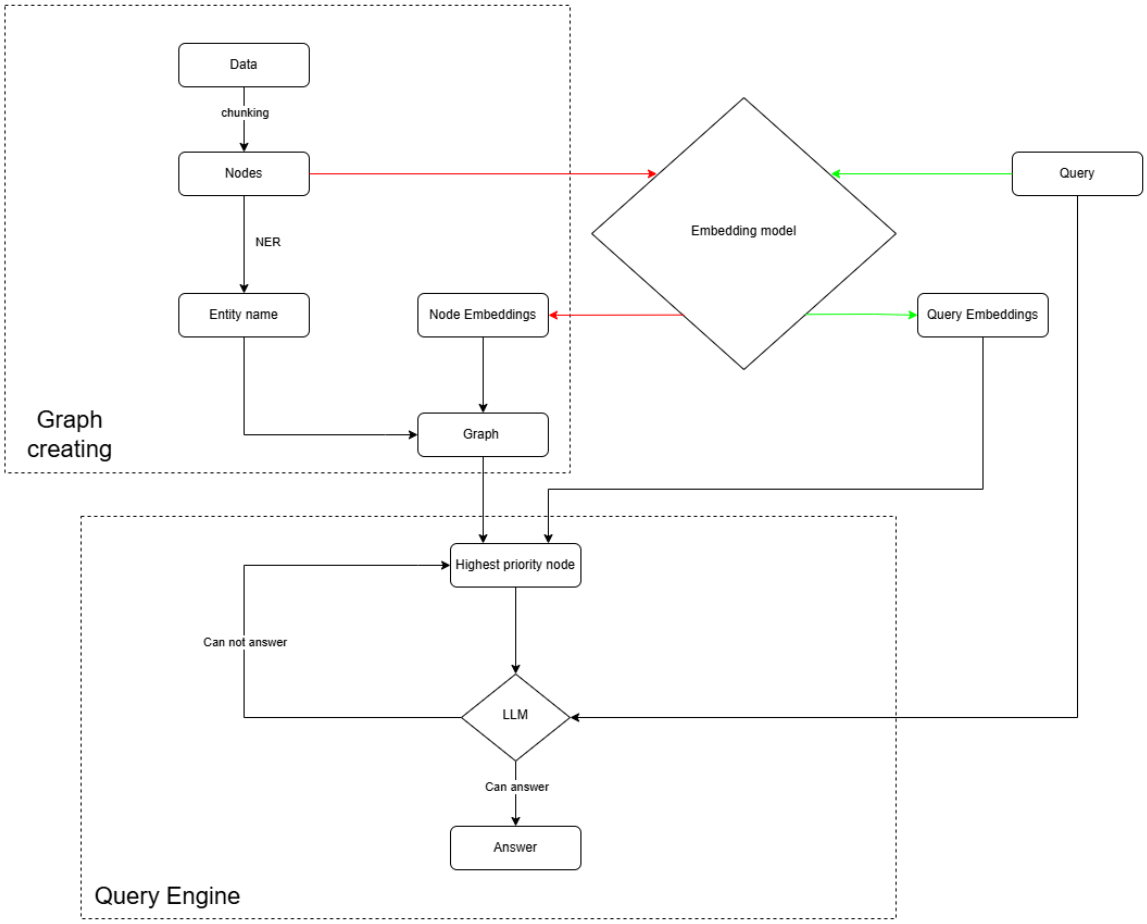


Figure 7: Graph-RAG structure

The original Graph-RAG papers highlighted several limitations, such as high token costs, the retrieval of irrelevant data, and inefficient graph construction. To address these issues, our Graph-RAG framework introduces several improvements. We reduce token costs by optimizing the chunking and embedding processes, enhance the relevance of retrieved data through advanced filtering mechanisms, and refine the graph construction methodology to more accurately represent the underlying relationships in the data. These enhancements contribute to a more efficient and accurate query-answering process, mitigating the drawbacks identified in earlier works.

The workflow of our Graph-RAG structure is illustrated in Figure 7. The system, as depicted in the diagram, represents a hybrid query-answering framework that integrates graph-based reasoning with embedding models and a large language model (LLM) to enhance response generation. The process begins with the preprocessing of input data into smaller, manageable chunks using an agentic-like chunking approach. This method ensures that the data is divided into goal-driven chunks suitable for subsequent retrieval tasks. From these chunks, entity names are extracted using the spaCy library, and nodes are created to represent the underlying data points. Once the nodes are generated, a Vietnamese embedding model is employed to produce vector representations (embeddings) for each node, facilitating semantic matching. A graph is then constructed by linking these nodes based on their relationships, which are

determined by entity names and their similarity in embedding space, measured via cosine similarity.

When a query is received, the embedding model generates query embeddings, which are compared with the node embeddings to identify the most relevant nodes. The data from the highest-priority node is incorporated into the prompt along with the query to facilitate answer generation based on the provided data. If this node contains sufficient information to answer the query, a response is directly generated. Otherwise, the system considers the next highest-priority node to expand the context for the LLM. This process continues iteratively, traversing through the nodes until either all relevant nodes are examined or an answer is obtained.In cases where the model is unable to answer the query based on the node data, the query is forwarded to the LLM. The LLM then attempts to generate a response based on the contextual information derived from the graph. This integrated approach, combining graph-based search for structural understanding and LLM reasoning for complex queries, establishes a robust mechanism for query answering, effectively utilizing both structured and unstructured data.

3.2.1 Vietnamese Embeddings

Computers do not inherently understand natural language; they process numerical data such as vectors and matrices. To facilitate this, text is represented in vector form, known as embeddings, ensuring that words with similar meanings are mapped to similar vectors. Various pre-trained word embeddings, such as GloVe, FastText, and Gensim Word2Vec, are commonly used. Additionally, OpenAI offers advanced embeddings, such as text-embedding-ada-002. In our approach, we utilize the GPT-4 mini model as our reasoning model after extracting information from the knowledge graph. Initially, we also explored embedding models from the OpenAI platform to ensure consistency, as we aimed to use a unified source and technology stack from the same provider. However, despite the ease of implementation, these embeddings may not be cost-effective for larger datasets due to the need for tokens during both the data ingestion process and subsequent queries.

Therefore, we decided to explore more alternative embedding models and lean toward Vietnamese embedding models. They are mostly free and provide accurate embeddings for our historical dataset. Among these, we selected the Vietnamese-embedding model developed by Dang Van Tuan (dangvantuan/vietnamese-embedding). This framework is a custom-designed sentence embedding model tailored specifically to the Vietnamese language, built upon the powerful capabilities of PhoBERT, a pre-trained language model derived from the RoBERTa architecture. The model leverages PhoBERT to transform Vietnamese sentences into a 768-dimensional vector space, enabling diverse applications such as semantic search and text clustering. These embeddings encapsulate the nuanced meanings of Vietnamese text, capturing both lexical and contextual elements of the language. The only downside of this embeddings model is the limited number of words it can embed at once. However, considering that the pros outweigh the cons, we decided to adapt to the model's limitations.

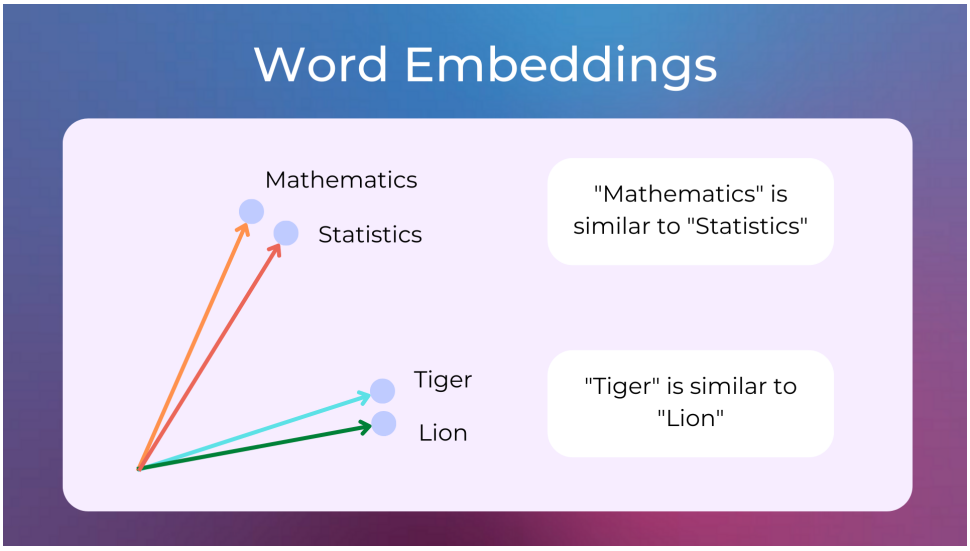


Figure 8: Embedding example[19]

Complete Model Architecture[20]:

```
SentenceTransformer(
  (0): Transformer({'max_seq_length': 512, 'do_lower_case': False})
       with Transformer model: RobertaModel
  (1): Pooling({'word_embedding_dimension': 768,
                'pooling_mode_cls_token': False,
                'pooling_mode_mean_tokens': True,
                'pooling_mode_max_tokens': False,
                'pooling_mode_mean_sqrt_len_tokens': False,
                'pooling_mode_weightedmean_tokens': False,
                'pooling_mode_lasttoken': False,
                'include_prompt': True})
```

Model	Pearson correlation	Spearman correlation	#params
dangvantuan/vietnamese-embedding	88.33	88.20	135M
VoVanPhuc/sup-SimCSE-VietNamese-phobert-base	84.65	84.59	135M
keepitreal/vietnamese-sbert	84.51	84.44	135M
bkai-foundation-models/vietnamese-bi-encoder	78.05	77.94	135M

Figure 9: Comparison between top Vietnamese embedding models [20]

3.2.2 Document Processor

This class splits documents into smaller chunks. Each chunk has its own embeddings, and these embeddings are compared using cosine similarity to determine which chunks are relevant to each other. The class has two main hyperparameters:

- chunk_size: Sets the number of characters in a chunk (must be fewer than 512 due to the model we use).

- chunk_overlap: Specifies the number of characters at the beginning of a new chunk that overlap with the previous chunk (within the interval [0, 500]).

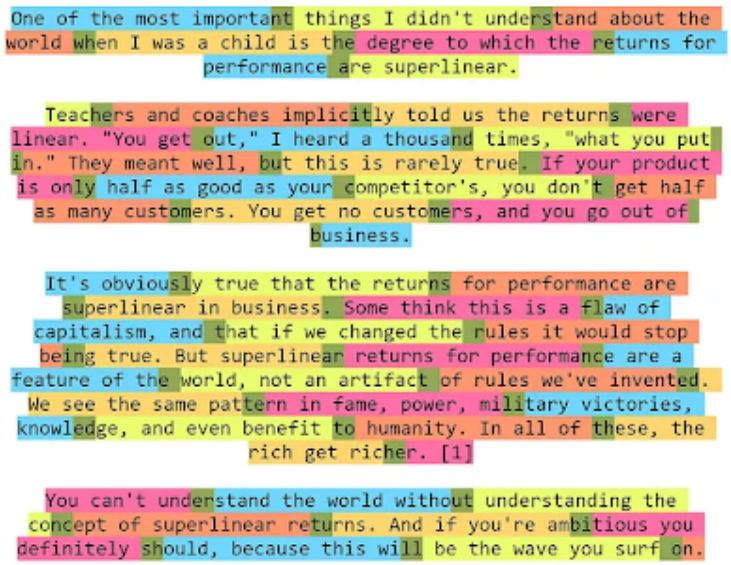


Figure 10: chunk_size and chunk_overlap [21] (different color interpret different chunk and green color interpret the overlap part)

Because of the way we format the data, the two adjacent chunks are more likely to correlate. Therefore, when using RAG to find information about a topic, it should consider nearby chunks to gather additional context. Instead of cutting a chunk in the middle of a word, we decided to split it after a complete word or punctuation mark. This approach maximizes information retention and minimizes the difference in the number of words between chunks. If we were to split a chunk by breaking a line, the differences could be significant, and the results might become less efficient.

3.2.3 KnowledgeGraph

This is where we turn out chunks of text into a graph.There are 2 main ways to approach this:

- Create a graph with nodes that are people, special events and places and the edges reflect the relationships between those nodes.This method is better when we have a large dataset and very costly. It is usually suitable fo healthcare topics like proteins,viruses or diseases. We speculate that historical data might work as well as healthcare data, but given that there are no paper related to applying this method to historical domain, we decided to go with the second approach.

- Each chunk that we turn the data into in the last class will become a node Because nodes are now not people,events and places they have no relation to each other other than their

meaning as a whole and their similarity in words.Normally,other RAG methods will use an embedding model to create embeddings for each chunk and comparing them using cosine similarity, create a similarity matrix for the embeddings.If the cosine similarity is larger than a threshold of some sort, an edge will be created ,reflecting the relationship between two nodes.This method is solid,but our testing show that it is not enough.Therefore we combined it with extracting concepts and entities.

Entities are extracted using spaCy in four categories:

- PERSON Names of individuals.

- ORG Organizations.

- GPE Geo-political entities (e.g., countries, cities,...).

- WORK_OF_ART Creative works (e.g., books, paintings,...).

spaCy is a Python library designed for performing a variety of Natural Language Processing (NLP) tasks, including Named Entity Recognition (NER), text classification, part-of-speech tagging, and more. Entity extraction, in the context of search, involves identifying which specific fields in a query should be targeted instead of querying all fields indiscriminately. This helps enhance precision in search results. For instance, consider the query Galaxy S21. Entity extraction can determine that the user's intent is to search for brand:Samsung AND model:S21 rather than retrieving unrelated results like telescopes or astronomical phenomena named "Galaxy."

Concepts are extracted using LLM. By leveraging the PromptTemplate from langchain, we can create a prompt that use GPT 4o mini model to extract key concepts After that we now have similarity matrix and and shared concepts for nodes.In order for it to create graph we still use similarity threshold.However, the weight of the edges now is the combination of the shared concepts and similarity score.It is calculated as:

$$\alpha \cdot \textit{similarity_score} + \beta \cdot \textit{normalized_shared_concepts} \tag{3}$$

In this, we have 3 other hyperparameters:

- edge_threshold: Decreasing this will create more edges between nodes but might increase inference time and cost.

- alpha: Increasing this will make edges more dependent on the meaning.

- beta: Increasing this will make edges more dependent on the topics.

3.2.4 Query Engine

The QueryEngine class forms the backbone of our Retrieval-Augmented Generation (RAG) model, offering a sophisticated method for resolving complex queries. This system integrates three pivotal components: a vector store for dense retrieval, a knowledge graph for structured reasoning, and a language model (LLM) for natural language understanding and response generation. It achieves a balance between semantic similarity and graph-based reasoning, making it highly effective for intricate information retrieval tasks.

The workflow begins by retrieving semantically relevant documents from the vector store. These documents, identified using cosine similarity, serve as entry points for the knowledge graph traversal. The knowledge graph organizes information as interconnected nodes, and the system employs a modified Dijkstra's algorithm to explore this graph. The traversal prioritizes nodes based on connection strength (edge weights), ensuring that the most relevant and

strongly connected information is accessed first. During traversal, the system dynamically expands the context by accumulating content from visited nodes, while also tracking concepts to avoid redundancy and guide exploration towards new, related information.

Table 1: Knowledge Graph Nodes and Associated Information

Node ID	Content	Concepts	Neighbor Node ID	Edge Weights
1	The Dien Bien Phu campaign was a decisive battle during the First Indochina War.	Dien Bien Phu, First Indochina War	2, 3	0.9, 0.8
2	The Viet Minh forces surrounded the French at Dien Bien Phu in early 1954.	Viet Minh, French	1, 4	0.8, 0.7
3	This victory marked the end of French colonial rule in Vietnam.	Victory, End of colonial rule	1, 4	0.85, 0.6
4	Following Dien Bien Phu, the Geneva Accords led to the division of Vietnam into North and South.	Geneva Accords, Division of Vietnam	2, 3	0.75, 0.7

We begin with the initial node in the priority queue. The priority queue contains the most relevant node based on the query, which is Node 1 in this case. Its initial priority is calculated based on the similarity score.

$$\textit{Priority Queue} = \{(0.95, \textit{Node 1})\} \tag{4}$$

After Node 1 is explored. Its neighbors are identified and evaluated based on their edge weights. For each neighbor, the priority is computed as the sum of the current node's priority and the edge weight.

The priority of Neighbor 2 is calculated as:

$$\textit{Priority}_2 = \textit{Priority}_1 + \textit{Edge Weight}_{1\to 2} = 0.95 + 0.9 = 1.85 \tag{5}$$

The priority of Neighbor 3 is calculated as:

$$\textit{Priority}_3 = \textit{Priority}_1 + \textit{Edge Weight}_{1\to 3} = 0.95 + 0.8 = 1.75 \tag{6}$$

Based on the edge weights, the neighbors are added to the priority queue. The updated queue contains Node 2 and Node 3, sorted by their priorities.

$$\textit{Updated Priority Queue} = \{(1.85, \textit{Node 2}), (1.75, \textit{Node 3})\} \tag{7}$$

The next node to explore is the one with the highest priority. In this case, Node 2 has the highest priority of 1.85.

$$\textit{Next Node to Explore} = \textit{Node 2} \tag{8}$$

The process continues by exploring Node 2 and repeating the steps of evaluating its neighbors and updating the priority queue.

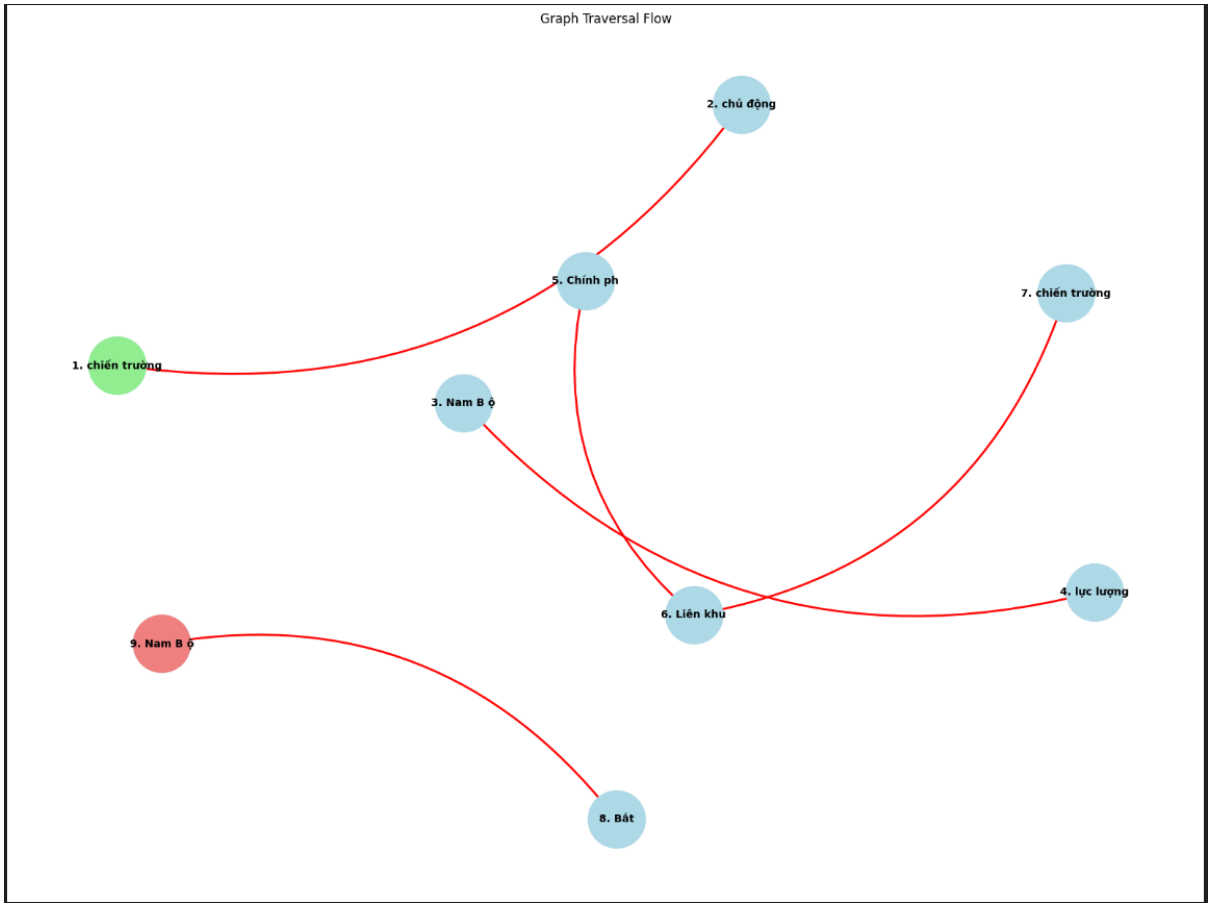


Figure 11: Our Dijkstra like algorithm

As the traversal proceeds, the system continually evaluates whether the accumulated context sufficiently answers the query. A custom validation chain, built with a prompt template, assesses the completeness of the gathered information. For example, if the system detects that the context describes logistics but lacks discussion on the strategic encirclement of the French forces, the traversal continues. Once a complete context is identified, the system halts further exploration, conserving resources.If the traversal fails to yield a complete answer, the LLM synthesizes a response based on the partial context.

To ensure accuracy, the QueryEngine incorporates an answer validation mechanism. A custom chain, created using a prompt template, determines whether the current context provides a complete answer to the query. If the context is sufficient, the system directly returns the answer; otherwise, the traversal continues. In cases where the graph traversal does not yield a satisfactory result, the LLM generates an answer based on the accumulated context. This approach balances efficiency and accuracy by minimizing unnecessary LLM invocations while ensuring that the output is both complete and relevant.

The system's design also accounts for the limitations of LLMs, such as token constraints. By dynamically building context and stopping traversal as soon as a complete answer is identified, the QueryEngine optimizes resource utilization. Additionally, hyperparameters like chunk_size and connection weights allow for fine-tuning, making the system adaptable to various datasets and domains.

IV Experimental result

4.1 Metrics

Given the inherent limitations of automatic metrics, human evaluation plays a critical role in assessing the performance of Graph-RAG models, particularly for tasks involving historical or domain-specific data. To evaluate the models, we use 3 different metrics:

- Factual Accuracy: The most significant advantage of human evaluation in the context of Graph-RAG models is the ability to assess factual accuracy. Unlike automatic metrics, which are limited to comparing surface-level similarities, human evaluators can verify whether the generated content accurately represents historical facts, events, and timelines. For instance, a Graph-RAG model may generate a response that is semantically coherent but historically inaccurate. Human evaluators can identify such discrepancies and ensure that the output aligns with established historical knowledge.

- Contextual Relevance and Coherence: In tasks involving historical knowledge, the accuracy of the generated content is not only about individual facts but also about how well those facts fit into a coherent and contextually relevant narrative. Human evaluators can assess whether the generated text maintains logical flow and consistency, ensuring that the historical events are presented in a way that makes sense both in isolation and within the broader historical context. This is particularly important for Graph-RAG models, which must generate outputs that are consistent with both the knowledge graph and the intended query.

- Overall Quality and User Experience: Finally, human evaluators provide valuable insights into the overall quality and fluency of the generated content. While automatic metrics can measure linguistic properties such as precision and recall, human evaluators can consider aspects such as readability, engagement, and clarity. In the case of historical texts, this also includes evaluating whether the model effectively communicates complex historical concepts in an accessible and coherent manner.

4.2 Training and Inference Setup

4.2.1 Hardware

In this study, we utilized our personal laptop as the primary computational resource to execute the experiments, which involved creating the graph and inference processes for the GraphRAG model. The laptop was equipped with an 11th Gen Intel(R) Core(TM) i7-11370H processor and 16GB of RAM, which provided sufficient computational power and memory for the tasks at hand. Despite the potential complexity of integrating graph-based knowledge representations with retrieval-augmented generation, the implementation ran smoothly without any issues related to RAM or memory limitations. The graph creation and inference processes were efficient and manageable within the available system resources. This demonstrates that GraphRAG can be effectively deployed on a mid-range personal computing device for experiments involving moderately sized datasets, underscoring its accessibility and adaptability for researchers without access to high-performance computing infrastructure.

4.2.2 Training duration

The process of creating the graph consists of three main steps: splitting chunks, extracting concepts, and adding edges. The time required for the splitting chunks step increases with the size of the dataset; for our dataset, this process takes approximately one hour. The

concept extraction step involves using named entity recognition to identify key entities. While using spaCy alone for NER would result in shorter processing times, we also leverage a large language model for enhanced entity extraction. However, employing the LLM through a batch API on a Tier 1 OpenAI account extended this step to 12 hours due to token usage limitations. To address this inefficiency, we upgraded to a Tier 2 OpenAI account, which allows for higher token usage per hour, reducing the processing time for this step significantly to approximately 30 minutes. Finally, the adding edges step is computationally lightweight and completes in about 2 minutes. This optimized pipeline enables efficient graph construction while maintaining high-quality entity and relationship extraction.

4.2.3 Tunning hyperparameters

Tuning the hyperparameters—alpha, beta, chunk_size, chunk_overlap, and edge_threshold—in our graph-based RAG (retrieval-augmented generation) model presents a unique challenge, as there are no predefined metrics or straightforward evaluation methods to determine the optimal settings. These hyperparameters directly influence the structure and connectivity of the knowledge graph, which in turn impacts the model's ability to retrieve relevant context and generate coherent responses. To address this, we employ an empirical approach, systematically experimenting with different combinations of hyperparameter values and analyzing the resulting outputs. The performance of each configuration is evaluated based on two primary criteria: token usage efficiency and the quality of the generated answers. Token usage provides insight into resource efficiency, while the quality assessment focuses on how well the output aligns with task-specific requirements, such as relevance, coherence, and informativeness. Through this iterative process, we aim to identify the hyperparameter settings that strike a balance between computational efficiency and answer quality, ultimately optimizing the model's performance.

After tuning various hyperparameters in our graph-based RAG model, several key insights have emerged. The alpha and beta parameters are critical in determining the relationship between nodes, as their ratio directly influences edge weights based on either semantic meaning or keyword matching, as defined by our formula. chunk_size and chunk_overlap, while less sensitive to minor variations, must follow a specific rule: chunk_size should be 2-5 times larger than chunk_overlap. A chunk_overlap close to the chunk_size increases inference time, while a very small overlap reduces the relational coherence between chunks. edge_threshold plays a pivotal role in controlling node connections during the retrieval step. A threshold set too low causes the model to traverse an excessive number of nodes, increasing computational cost without improving relevance. Conversely, a threshold set too high may terminate the search prematurely, relying on unrelated information from the language model instead of leveraging graph data effectively. Balancing these parameters is essential to optimize both the efficiency and accuracy of the model.

Table 2: Comparison of Hyperparameter Configurations and Token Costs of the same question and answer

Configuration	Alpha	Beta	chunk_size	chunk_overlap	edge_threshold	Token Cost
Config 1	0.8	0.2	400	100	0.8	35612
Config 2	0.8	0.2	400	100	0.85	2231

V Results and Discussion

We evaluated our Graph-RAG model and GPT-based outputs using three human-assessed metrics: **Factual Accuracy**, **Contextual Relevance and Coherence**, and **Overall Quality and User Experience**. Each criterion was rated on a scale of 1 to 5, allowing for detailed comparisons of model performance across hyperparameter configurations and against GPT-based outputs.

5.1 Token Efficiency Analysis Across Hyperparameter Configurations

In this section, we analyze the impact of varying hyperparameters — alpha, beta, chunk_size, chunk_overlap, and edge_threshold — on the token usage of our Graph-RAG model. The results, summarized in Table 3, demonstrate significant variability in token efficiency based on these parameters. We have some key observations:

- edge_threshold Sensitivity: edge_threshold has a pronounced effect on token costs. A smaller threshold (e.g., 0.7) allows for extensive exploration of the graph, leading to excessive token usage (e.g., 114,023 tokens in Configuration 9). In contrast, a threshold of 0.85 optimizes the balance between graph exploration and token efficiency (e.g., 2937 tokens in Configuration 6).

- chunk_size and chunk_overlap: Configurations with smaller chunk_overlap relative to chunk_size (e.g., 400 vs. 100 in Configurations 5 and 6) exhibit lower token costs compared to those with higher overlaps (e.g., 500 vs. 200 in Configuration 10). This suggests that careful tuning of chunk parameters is essential for minimizing redundancy in the model's input processing.

- Alpha-Beta Ratio: Configurations maintaining a balanced ratio of alpha to beta (e.g., 0.8:0.2) achieve superior token efficiency across various chunk and edge_threshold settings. This indicates the importance of weighting semantic relevance (alpha) and keyword-based matching (beta) appropriately in the graph construction.

Summary of Results: Table 3 presents token usage for various hyperparameter configurations, highlighting the influence of each parameter. Additionally, Figure 12 illustrates the impact of edge_thresholds on token efficiency, while Figure 14 provides insights into the interplay between chunk_size and chunk_overlap.

Table 3: Token Usage for Various Hyperparameter Configurations

Config	Alpha	Beta	chunk_size	chunk_overlap	edge_threshold	Token Cost
1	1.0	0.0	500	200	0.8	5888
2	0.0	1.0	500	200	0.8	5533
3	0.7	0.3	500	200	0.8	6067
4	0.3	0.7	500	200	0.8	5907
5	0.8	0.2	400	100	0.8	3553
6	0.8	0.2	400	100	0.85	2937
7	0.8	0.2	500	100	0.8	4887
8	0.8	0.2	500	100	0.85	3396
9	0.8	0.2	500	100	0.7	114023
10	0.8	0.2	500	200	0.85	3637
11	0.8	0.2	500	200	0.9	3355

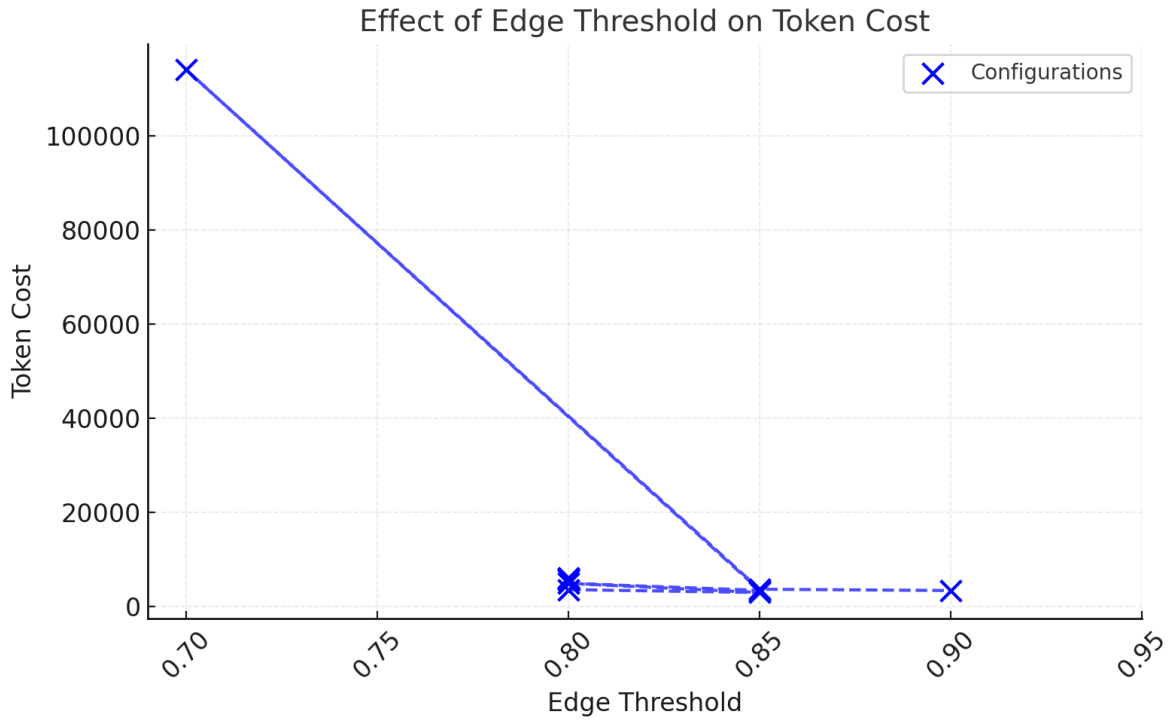


Figure 12: Effect of edge_threshold on Token Cost (chunk_size and chunk_overlap Fixed).

This analysis underscores the importance of fine-tuning hyperparameters to balance efficiency and output quality. The results emphasize the sensitivity of the model to edge_thresholds and the interplay between chunk_size and chunk_overlap, providing actionable insights for optimizing Graph-RAG implementations.

5.2 Hyperparameter Sensitivity Analysis

Table 4 highlights the impact of key hyperparameters—alpha, beta, chunk_size, chunk_overlap, and edge_threshold—on performance metrics and token usage. Configurations with lower edge_thresholds (e.g., 0.8) result in higher token costs, while a threshold of 0.85 balances efficiency and output quality.

Table 4: Evaluation Metrics for Different Parameter Configurations

Alpha	Beta	C.Size	C.Overlap	Edge	Fact	Rel. & Coh.	User Exp.	Tokens
0.8	0.2	400	100	0.8	4.859	4.93	4.766	3553
0.8	0.2	400	100	0.9	4.828	4.902	4.731	3355
0.8	0.2	500	100	0.8	4.884	4.932	4.78	4887
0.8	0.2	500	100	0.85	4.916	4.6	4.812	3398
0.8	0.2	400	100	0.85	4.874	4.92	4.766	2937
0.8	0.2	400	100	0.8	4.878	4.93	4.766	3553
0.8	0.2	500	200	0.85	4.859	4.92	4.768	3637

Figure 13 visualizes the sensitivity of overall quality to variations in edge_threshold, while Figure 14 presents a heatmap showing the interaction between chunk_size and chunk_overlap.

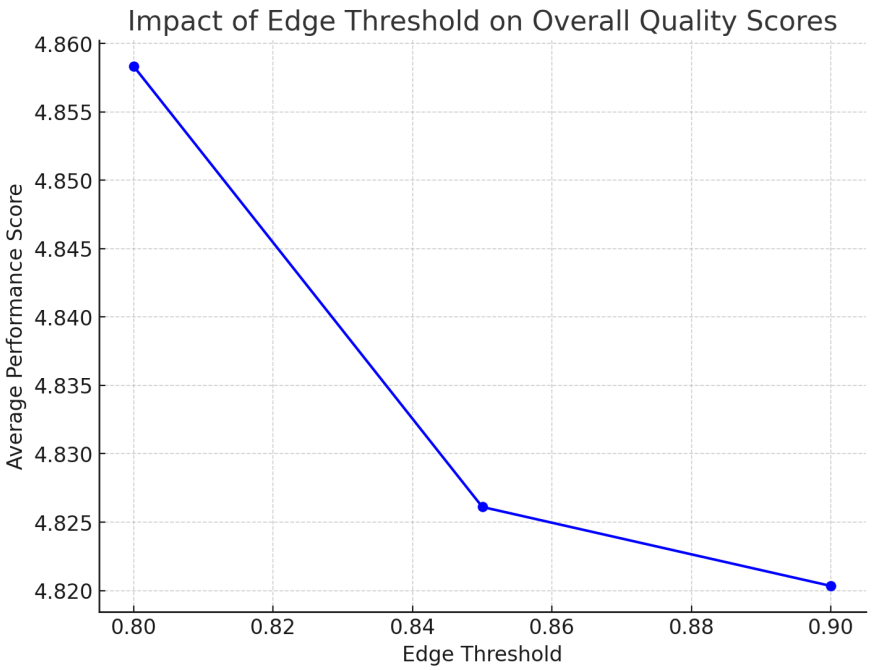


Figure 13: Line graph showing the impact of edge_threshold on overall quality scores.

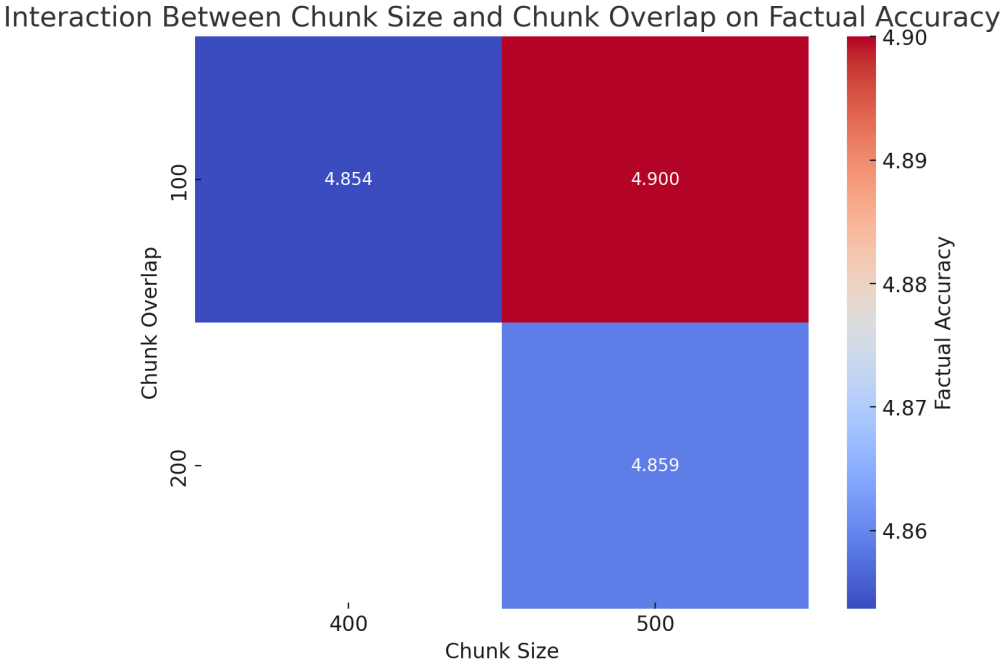


Figure 14: Heatmap showing the interaction between chunk_size and chunk_overlap on performance metrics.

5.3 Token Efficiency vs. Quality Tradeoff

In this analysis, we aim to select the best configuration based on the trade-off between *Weighted Quality Score* and *Token Usage*. The configurations provided are evaluated for efficiency using the metric:

$$Efficiency = \frac{Weighted\ Quality\ Score}{Token\ Usage} \tag{9}$$

where the *Weighted Quality Score* is calculated as the average of Factual Accuracy, Relevance and Coherence, and User Experience. The results for each configuration are shown in Table 5.

Configuration	**Efficiency** ($\frac{\text{Score}}{\text{Token Usage}}$)	**Token Usage**
Config 1	0.00137	3553
Config 2	0.00144	3355
Config 3	0.00099	4887
Config 4	0.00141	3398
Config 5	**0.00165**	2937
Config 6	0.00137	3553
Config 7	0.00133	3637

Table 5: Efficiency of Configurations Based on Weighted Quality Scores and Token Usage

From the table, the highest efficiency is achieved by **Config 5**, with an efficiency of 0.00165. This configuration is described by: Alpha= 0.8, Beta = 0.2 ,chunk_size = 400 ,chunk_overlap = 100, edge_threshold= 0.85

The selection of Config 5 is supported by its superior balance between achieving high-quality scores and minimizing token usage, thus maximizing efficiency.

5.4 Quantitative Comparison of Models

When evaluated across various question sets, our Graph-RAG model demonstrates superior factual accuracy compared to GPT-4. While our model lacks the advanced reasoning skills exhibited by GPT-4, it excels in providing specific, quantifiable data, as opposed to general or vague statements. This focus on precise numerical data enhances the clarity and reliability of the information presented. Moreover, our model proves more consistent in cases where GPT-4 tends to hallucinate or struggles to transfer relevant information to users. While GPT-4 may offer more sophisticated reasoning, the Graph-RAG model remains advantageous in delivering accurate and actionable data, particularly in scenarios that require precise and fact-based responses.

Table 6 shows the overall scores for our model and GPT-based outputs across the three evaluation metrics. Our Graph-RAG model achieves higher scores compare to GPT 4o mini in *Contextual Relevance and Coherence* demonstrating its reliability of the information presented

Table 6: Comparison of Model Performance Across Evaluation Metrics

Model	**Factual Accuracy**	**Relevance and Coherence**	**User Experience**
Our best Graph-RAG	4.874	4.92	4.766
GPT 4o	4.92	4.934	4.838
GPT 4o mini	4.858	4.93	4.766

GPT 4o Model Output	Our GraphRag Model Output
Sự tăng trưởng của mạng lưới đường sắt: Vào năm 1914, tổng chiều dài đường sắt ở Đông Dương đã lên đến hơn 4.000 km, chủ yếu phục vụ cho việc vận chuyển hàng hóa, đặc biệt là nông sản và khoáng sản từ các vùng sản xuất tới các cảng để xuất khẩu. Sự mở rộng các cảng và hệ thống tàu biển: Cảng Sài Gòn, Hải Phòng và các cảng khác đã được phát triển mạnh mẽ để phục vụ cho việc xuất khẩu hàng hóa ra thế giới. Các công ty vận tải biển, thuộc sở hữu của Pháp hoặc các công ty liên kết, gia tăng số lượng tàu để vận chuyển hàng hóa. Sự phát triển của vận tải đường bộ: Sự ra đời của các công ty vận tải đường bộ, đặc biệt là trong thập niên 1920-1930, đã giúp kết nối các thành phố lớn với các vùng nông thôn, phục vụ nhu cầu vận chuyển hàng hóa nội địa và tăng cường hoạt động thương mại. Thống kê về số lượng phương tiện vận tải: Các công ty vận tải lớn như Công ty vận tải đường sắt Đông Dương (Compagnie des Chemins de Fer de l'Indochine) đã có hàng nghìn đoàn tàu chạy trong khu vực, đồng thời số lượng xe tải, xe hơi trên các tuyến đường bộ cũng gia tăng mạnh mẽ. Mình	Sự phát triển của các công ty vận tải ở Đông Dương được phản ánh qua các số liệu sau: . Vốn đầu tư vào ngành vận tải: Vào năm 1930, vốn đầu tư vào ngành này là 100.000.000 francs. Trong giai đoạn 1888 - 1918, tổng vốn đầu tư vào ngành vận tải là 128 triệu francs. . Công ty vô danh hợp đồng dịch vụ vận tải đường biển được thành lập vào ngày 21-11-1921 với vốn 60.000.000 francs, quản lý và khai thác một đội tàu gồm 66 chiếc, tổng lượng tải 480.000 tấn, sức chở 49.000 tấn. . Sự gia tăng số lượng tàu của Công ty Bạch Thái Bư ở: từ 3 tàu thuê vào năm 1909 đã tăng lên 15 tàu vào năm 1915 và 30 tàu vào năm 1919. . Sự mở rộng các tuyến hàng không quốc tế, với hãng Air Vietnam liên kết với các hãng hàng không Mỹ, Pháp, Campuchia, Hồng Kông và Băng Cốc. . Năm 1952, Mỹ hùn vốn 65% vào công ty hàng không Việt Nam (Air Vietnam) Những số liệu này cho thấy sự phát triển đáng kể của ngành vận tải ở Đông Dương trong các giai đoạn khác nhau

Figure 15: Example of our model answer compare to GPT 4o

Figure 15 compares two models' outputs, showcasing the strengths of each in explaining historical events.The GPT-4 model offers a detailed and coherent narrative, demonstrating strong reasoning and a sophisticated understanding of the topic. Its explanation connects historical trends with logical clarity, making the analysis feel comprehensive. However, it lacks specific numerical data to reinforce its points. In contrast, the Graph RAG model provides a more data-driven explanation, incorporating precise figures, such as noting that in 1930, investment in the transportation sector was 100,000,000 francs, and between 1888 and 1918, total investment in the sector was 128 million francs. While its reasoning is less elaborate, the use of concrete data lends the explanation greater empirical support. Thus, the Graph RAG output offers more measurable insight, grounding the analysis in factual details, while GPT-4 excels in articulating broader connections and logical flow.

VI Conclusion and Improvement

6.1 Conclusion

This study implemented a Graph-RAG architecture that combines Named Entity Extraction with a Large Language Model (LLM) to extract key concepts for answering historical questions. Graph-RAG introduces a novel approach to connecting nodes by utilizing entity names and key concepts, employing a Dijkstra-like algorithm for graph traversal, and applying a distinct prompting technique to provide context for the LLM. This architecture demonstrates superior accuracy compared to the GPT-4 mini model and excels in answering domain-specific questions, particularly in the field of Vietnamese history.

6.2 Improvement

During testing, we found that the model is not compatible with several embedding models, limiting our ability to perform comprehensive comparisons to identify the most suitable one. Additionally, we encountered an issue with saving the model to a pickle file, meaning that if we were to rerun the code, it would take a considerable amount of time to rebuild the graph from scratch. As a workaround to save time, we have saved the key concepts of each node in a JSON file; however, this method is not as efficient as desired.

As for future work, We have developed plans to enrich the dataset as it currently relies on just over three primary sources. To address the saving issue, we aim to store our graph data in Neo4j or NebulaGraph. Furthermore, we anticipate improving model performance by better expressing the relationships between nodes through alternative methods. Embedding comparison methods can also be impprove by using a Hybrid Metric (Cosine + Semantic Fine-Tuning) to retrieving the most relevant nodes from the graph.

References

[1] Wayne Xin Zhao et al. *Dense Text Retrieval based on Pretrained Language Models: A Survey*. 2022. arXiv: 2211.14876 [cs.IR]. URL: https://arxiv.org/abs/2211.14876.

[2] Sewon Min et al. *Knowledge Guided Text Retrieval and Reading for Open Domain Question Answering*. 2020. arXiv: 1911.03868 [cs.CL]. URL: https://arxiv.org/abs/1911.03868.

[3] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL]. URL: https://arxiv.org/abs/1810.04805.

[4] Yinhan Liu et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. arXiv: 1907.11692 [cs.CL]. URL: https://arxiv.org/abs/1907.11692.

[5] Mandar Joshi et al. *SpanBERT: Improving Pre-training by Representing and Predicting Spans*. 2020. arXiv: 1907.10529 [cs.CL]. URL: https://arxiv.org/abs/1907.10529.

[6] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165.

[7] Colin Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. 2023. arXiv: 1910.10683 [cs.LG]. URL: https://arxiv.org/abs/1910.10683.

[8] Mike Lewis et al. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. 2019. arXiv: 1910.13461 [cs.CL]. URL: https://arxiv.org/abs/1910.13461.

[9] Jean Nyandwi. *The Transformer Blueprint: A Holistic Guide to the Transformer Neural Network Architecture*. Available at deepprevision.github.io. 2023. URL: https://deepprevision.github.io/posts/001-transformer/.

[10] Kelvin Guu et al. *REALM: Retrieval-Augmented Language Model Pre-Training*. 2020. arXiv: 2002.08909 [cs.CL]. URL: https://arxiv.org/abs/2002.08909.

[11] Ian A. Crawford. *Science enabled by a Moon Village*. 2017. arXiv: 1706.06698 [astro-ph.EP]. URL: https://arxiv.org/abs/1706.06698.

[12] Gautier Izacard and Edouard Grave. *Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering*. 2021. arXiv: 2007.01282 [cs.CL]. URL: https://arxiv.org/abs/2007.01282.

[13] Sara Rosenthal et al. *CLAPNQ: Cohesive Long-form Answers from Passages in Natural Questions for RAG systems*. 2024. arXiv: 2404.02103 [cs.CL]. URL: https://arxiv.org/abs/2404.02103.

[14] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2021. arXiv: 2005.11401 [cs.CL]. URL: https://arxiv.org/abs/2005.11401.

[15] Gautier Izacard et al. *Atlas: Few-shot Learning with Retrieval Augmented Language Models*. 2022. arXiv: 2208.03299 [cs.CL]. URL: https://arxiv.org/abs/2208.03299.

[16] Diego Sanmartin. *KG-RAG: Bridging the Gap Between Knowledge and Creativity*. 2024. arXiv: 2405.12035 [cs.AI]. URL: https://arxiv.org/abs/2405.12035.

[17] Diego Sanmartin. *KG-RAG: Bridging the Gap between Knowledge and Creativity*. Available at dsanmart.github.io. 2024. URL: https://dsanmart.github.io/KG-RAG/.

[18] Garima Agrawal et al. *Can Knowledge Graphs Reduce Hallucinations in LLMs? : A Survey*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico, June 2024. DOI: 10.18653/v1/2024.naacl-long.219. URL: https://aclanthology.org/2024.naacl-long.219.

[19] NLPlanet. *Text as Vectors (Embeddings)*. 2024. URL: https://www.nlplanet.org/course-practical-nlp/01-intro-to-nlp/11-text-as-vectors-embeddings.

[20] Van Tuan Dang. *Vietnamese Embedding Model*. Available at Hugging Face. 2021. URL: https://huggingface.co/dangvantuan/vietnamese-embedding.

[21] Ashish Abraham. "How Document Chunk Overlap Affects a RAG Pipeline". In: *AI Advances* (July 2024). URL: https://ai.gopubby.com/how-document-chunk-overlap-affects-a-rag-pipeline-9b8931845c20.